

PI PROP INTELLIGENCE

INFRASTRUCTURE GUIDE

Domains, platforms, data sources, security boundaries, deployments, and recovery

CURRENT EDITION Updated through August 21, 2026. Web/PWA release; no App Store or Play Store distribution is assumed.

Prepared for PI PROP INTELLIGENCE LLC

Independent sports research and intelligence only. PI does not facilitate wagering or accept bets.

Contents

- 1. Architecture**
- 2. Production sites and dashboards**
- 3. Customer-facing prop sites**
- 4. Data and content providers**
- 5. Application services**
- 6. Data flow and freshness**
- 7. Security boundary**
- 8. Deployment and verification**
- 9. Monitoring and incident response**
- 10. Backup and recovery**
- 11. Known boundaries**

1. Architecture

The production platform is a Flutter web/PWA client served through Vercel and a FastAPI/Python backend served through Render. Supabase provides authentication and PostgreSQL data services. Redis and RQ provide shared cache, queueing, coordination, rate limits, and short-lived operational state. External providers supply markets, scoreboards, statistics, images, billing, CAPTCHA, and notifications.

Layer	Service	Responsibility
Public web	propsintell.com	Marketing, product, pricing, legal, installation, contact, and authentication entry points.
Application	app.propsintell.com / Vercel	Flutter web/PWA, responsive shell, browser-delivered public configuration, and release caching.
API	api.propsintell.com / Render	Authentication enforcement, prop catalog, models, grading, provider orchestration, webhooks, and owner telemetry.
Identity/data	Supabase	Authentication, profiles, PostgreSQL, row-level security, storage, and managed backups.
Cache/jobs	Redis + RQ on Render	Distributed catalog, queues, workers, cooldowns, rate limiting, image maps, and recovery state.
Source/release	GitHub	Repository, commit history, CI, release source of truth, and deployment triggers.

2. Production sites and dashboards

Site	Use	Owner verification
Vercel	Frontend deployment and public environment variables	Latest main deployment is Ready; custom domain points to it; update prompt serves the intended bundle.
Render	API, worker, Redis, cron, secrets, and logs	Services Live; deploy commit matches GitHub; workers/cron healthy; no restart loop.
Supabase	Auth, PostgreSQL, RLS, backups	Owner MFA; redirects; Security Advisor; backups/PITR; no exposed service role.
GitHub	Code, commits, Actions	main traceable; checks understood; no secrets; deployment source matches intended commit.
RevenueCat	Products, offerings, entitlements, webhook	Core/Pro mapping valid; webhook authenticated; sandbox and production separated.

Site	Use	Owner verification
OneSignal	Push delivery and device tags	App ID and backend credentials configured; permission and test notification verified.
Cloudflare Turnstile	Authentication bot protection	Site key covers app domain; secret remains server-side/Supabase; hostname is correct.

3. Customer-facing prop sites

Prop site	Normalized label and behavior
PrizePicks	Pick'em prop inventory. PI displays provider lines and tracks research selections; confirm rules in PrizePicks.
Underdog	Fantasy pick'em inventory and tracked slip presentation; provider rules and multipliers remain authoritative.
FanDuel	Sportsbook prop and game-market inventory where licensed data is available; verify price/odds in FanDuel.
DraftKings Pick6	Pick6 prop inventory, separate from DraftKings sportsbook lines.
Draft Picks	Normalized provider label used when this feed returns inventory; do not confuse it with DraftKings.
Betr	Betr Picks inventory where the connected feed provides active markets.

Provider separation is a presentation and trust requirement: props from different sites are grouped under separate provider headers and are not mixed into one indistinguishable card list. DraftKings Pick6 and sportsbook DraftKings are distinct products.

4. Data and content providers

Provider	Role	Protection / fallback
The Odds API	Sportsbook events, player props, game markets, quota telemetry	Quota reserve, nearest-event priority, adaptive cooldown, cache, stale disclosure, and provider health.
SportsGameOdds	Supplemental prop events and provider inventory	Retry-After, exponential cooldown, health counters, and no request cascade during rate limits.
API-Sports	Selected scoreboard, player statistics,	Sport-aware fallback, cache, and explicit

Provider	Role	Protection / fallback
	and grading support	provider error reporting.
ESPN	Scoreboard/stat context and player images	Redis/bundled headshot maps and source-specific fallback.
Sportmonks	Soccer history, player context, and images	Shared cache plus local fallback; coverage depends on subscribed competition.
Official league APIs	MLB and supported official context	Cached scheduled ingestion; request-time paths stay bounded.

5. Application services

- Flutter/Dart: responsive board, scoreboards, research overlays, builders, history, chat, account, billing, and owner operations.
- FastAPI/Python: API routing, token validation, authorization, normalization, modeling, line history, grading, provider health, webhooks, and telemetry.
- PostgreSQL: user, profile, ticket, prediction, historical, officiating, line-snapshot, engagement, billing, and security-event records.
- Redis/RQ: compressed prop catalog, shared cache, queue coordination, retries, rate limits, provider cooldowns, and headshot maps.
- RevenueCat: Core and Pro entitlement state. Backend-verified webhook results control durable access.
- OneSignal: web push delivery. The browser permission and member/device association are separate prerequisites.

6. Data flow and freshness

1. Scheduled/live sync jobs fetch provider inventory with bounded concurrency and quota controls.
2. Provider payloads are normalized into one prop contract while preserving source site, event, player, market, and update time.
3. The catalog assigns stable group identities and player-image keys before publishing compressed shared state.
4. Every pregame line change is recorded in `sportsbook_line_snapshots`. The live catalog reapplies the earliest observation to expose genuine opening-to-current movement.
5. The API filters by sport, site, category, verdict, freshness, and access tier; the frontend must not reconstruct protected Pro direction.
6. Stale/no-inventory/outage states remain distinct. Owners see last successful provider updates and alerts when an expected source becomes stale.

Provider state	Definition	Operational response
Active	Fresh inventory returned	Monitor normally.

Provider state	Definition	Operational response
No Current Inventory	Provider healthy but no props matched the current schedule	Do not classify as outage.
Stale	Last successful inventory exceeds freshness threshold	Inspect sync, quota, cooldown, cache, and worker.
Outage	Expected provider failed or cannot be reached	Use fallback/cached disclosure and open incident follow-up.

7. Security boundary

NEVER IN THE BROWSER Provider secrets, Supabase service-role key, database connection strings, Redis URL, RevenueCat webhook authorization, private signing secrets, deployment tokens, or user access tokens in logs.

- Allowed public configuration includes Supabase publishable/anon key, Turnstile site key, RevenueCat public SDK key, OneSignal public app ID, and API base URL.
- The backend validates Supabase tokens and independently enforces owner, admin, Core, and Pro access.
- Database row-level security remains enabled even when server routes also authorize access.
- CORS permits approved production origins. Rate limiting and safe errors protect public routes.
- Logs omit authorization headers, complete billing objects, secrets, and unnecessary personal data.
- Owner Ops is owner-only. Admin/tester access is not interchangeable with owner authorization.

8. Deployment and verification

1. Review intended files and test results; exclude unrelated or generated artifacts.
2. Commit to main with a descriptive message and push GitHub.
3. Confirm Vercel and Render deploy the same intended commit.
4. Verify public site and app return successfully; update the PWA when the release prompt appears.
5. Verify /ready dependencies, prop catalog, auth protection, provider state, scoreboard, headshots, filters, active slip, research overlay, and owner operations.
6. Record the commit, deployment timestamps, known warnings, and rollback candidate.

9. Monitoring and incident response

Symptom	First check	Next action
App fails to load	Vercel deployment/domain	Inspect build, custom domain, public variables, and PWA cache.

Symptom	First check	Next action
App loads; props fail	Owner Ops and Render API	Check release, catalog source, providers, quota, Redis, queue, and freshness.
Stale provider alert	Provider last success and cooldown	Do not force-refresh through rate limit; verify recovery/fallback.
Photos fail	Headshot cache/source	Confirm stable keys, mappings, scheduled image sync, and fallback behavior.
Login/CAPTCHA fails	Supabase Auth and Turnstile	Check site/secret pairing, hostnames, redirects, and auth limits.
Paid access missing	RevenueCat then Supabase	Verify entitlement, webhook authentication/idempotency, and profile tier.
Tickets unsettled	Owner grading review	Confirm final game state, event identity, result source, and reconciliation.

10. Backup and recovery

- Confirm Supabase backup retention and point-in-time recovery policy monthly.
- Preserve durable ticket, prediction, line-history, and migration-checksum tables through code releases.
- Redis is operational state, not the only durable source for critical user history.
- Recover code by redeploying a known healthy commit; never delete production data to fix a frontend release.
- Test restore procedures and document ownership before an incident, not during one.

11. Known boundaries

- NFL officiating profiles are not currently implemented; Officiating Tracker supports NBA, WNBA, and MLB.
- Provider and sport inventory changes with schedule, subscription coverage, geography, and upstream market release timing.
- The web/PWA is the supported distribution path for this edition; App Store and Play Store release procedures are out of scope.
- No browser application can hide downloaded client code. Proprietary logic, secrets, and protected payloads remain server-side.